

1. Alternate Printing (Even-Odd) • Create two threads: one prints even numbers, another prints odd numbers (1–100) in sequence.

```
1 package assignment;
2 public class EvenOddPrinter {
3     private static int counter = 1;
4     private static final int LIMIT = 100;
5     private static final Object lock = new Object();
6     public static void main(String[] args) {
7         Thread oddThread = new Thread() -> {
8             while (counter <= LIMIT) {
9                 synchronized (lock) {
10                     if (counter % 2 == 0) { // If even, wait
11                         try { lock.wait(); } catch (InterruptedException e) {}
12                     }
13                     if (counter <= LIMIT) {
14                         System.out.println("Odd: " + counter++);
15                     }
16                     lock.notify();
17                 }
18             }
19         };
20         Thread evenThread = new Thread() -> {
21             while (counter <= LIMIT) {
22                 synchronized (lock) {
23                     if (counter % 2 != 0) { // If odd, wait
24                         try { lock.wait(); } catch (InterruptedException e) {}
25                     }
26                     if (counter <= LIMIT) {
27                         System.out.println("Even: " + counter++);
28                     }
29                     lock.notify();
30                 }
31             }
32         };
33         oddThread.start();
34         evenThread.start();
35     }
36 }
```

Console Output:

```
<terminated> EvenOddPrinter [Java Application] C:\Program Files\Java\jdk-26.0.1\bin\javaw.exe (2 May 2026, 1:12:57 pm – 1:12:57 pm elapsed 0:00:00) [pid: 15004]
Odd: 1
Even: 2
Odd: 3
Even: 4
Odd: 5
Even: 6
Odd: 7
```

2. Print A B C in Sequence • Three threads print A, B, C repeatedly in order (ABCABC...).

```
1 package assignment;
2 public class ABCSequence {
3     private int turn = 1; // 1 for A, 2 for B, 3 for C
4     private final int LIMIT = 10; // Number of times to print ABC
5     public synchronized void printA() {
6         for (int i = 0; i < LIMIT; i++) {
7             while (turn != 1) {
8                 try { wait(); } catch (InterruptedException e) {}
9             }
10            System.out.print("A");
11            turn = 2;
12            notifyAll();
13        }
14    }
15    public synchronized void printB() {
16        for (int i = 0; i < LIMIT; i++) {
17            while (turn != 2) {
18                try { wait(); } catch (InterruptedException e) {}
19            }
20            System.out.print("B");
21            turn = 3;
22            notifyAll();
23        }
24    }
25    public synchronized void printC() {
26        for (int i = 0; i < LIMIT; i++) {
27            while (turn != 3) {
28                try { wait(); } catch (InterruptedException e) {}
29            }
30            System.out.print("C ");
31            turn = 1;
32            notifyAll();
33        }
34    }
35    public static void main(String[] args) {
36        ABCSequence obj = new ABCSequence();
37        new Thread(obj::printA).start();
38        new Thread(obj::printB).start();
39        new Thread(obj::printC).start();
40    }
41 }
```

Console Output:

```
<terminated> ABCSequence [Java Application] C:\Program Files\Java\jdk-26.0.1\bin\javaw.exe (2 May 2026, 1:16:16 pm – 1:16:16 pm elapsed 0:00:00) [pid: 14360]
ABC ABC ABC ABC ABC ABC ABC ABC ABC
```

3. Bank Account System • Multiple threads deposit and withdraw from same account safely.

The screenshot shows the Eclipse IDE with the `BankSystem.java` file open. The code defines a `BankAccount` class with `deposit` and `withdraw` methods, and a `BankSystem` class with a `main` method that creates three threads (`User_1`, `User_2`, `User_3`) to perform deposits and withdrawals on a shared `BankAccount`.

```
1 package assignment;
2
3 class BankAccount {
4     private double balance = 1000;
5     public synchronized void deposit(double amount) {
6         balance += amount;
7         System.out.println(Thread.currentThread().getName() + " deposited " + amount + ". Balance: " + balance);
8     }
9     public synchronized void withdraw(double amount) {
10        if (balance >= amount) {
11            balance -= amount;
12            System.out.println(Thread.currentThread().getName() + " withdrew " + amount + ". Balance: " + balance);
13        } else {
14            System.out.println(Thread.currentThread().getName() + " failed: Insufficient funds.");
15        }
16    }
17
18    public class BankSystem {
19        public static void main(String[] args) {
20            BankAccount account = new BankAccount();
21
22            Runnable task = () -> {
23                account.deposit(200);
24                account.withdraw(500);
25            };
26            Thread user1 = new Thread(task, "User_1");
27            Thread user2 = new Thread(task, "User_2");
28            Thread user3 = new Thread(task, "User_3");
29            user1.start();
30            user2.start();
31            user3.start();
32        }
33    }
34 }
```

The console output shows the execution of the program:

```
<terminated> BankSystem (1) [Java Application] C:\Program Files\Java\jdk-26.0.1\bin\javaw.exe (2 May 2026, 1:28:42 pm - 1:28:42 pm elapsed 0:00:00) [pid: 13932]
User_1 deposited 200.0. Balance: 1200.0
User_1 withdrew 500.0. Balance: 700.0
User_2 deposited 200.0. Balance: 900.0
User_2 withdrew 500.0. Balance: 400.0
User_3 deposited 200.0. Balance: 600.0
User_3 withdrew 500.0. Balance: 100.0
```

4. Online Food Delivery System • Multiple orders processed by limited delivery agents (thread pool).

The screenshot shows the Eclipse IDE with the `FoodDeliverySystem.java` file open. The code defines a `DeliveryTask` class that implements `Runnable`, and a `FoodDeliverySystem` class that uses a `ThreadPoolExecutor` to manage a pool of delivery agents.

```
1 package assignment;
2 import java.util.concurrent.ExecutorService;
3 import java.util.concurrent.Executors;
4
5 class DeliveryTask implements Runnable {
6     private final int orderId;
7     public DeliveryTask(int id) { this.orderId = id; }
8     @Override
9     public void run() {
10        String agentName = Thread.currentThread().getName();
11        System.out.println("Agent " + agentName + " is picking up Order #" + orderId);
12        try { Thread.sleep(2000); } catch (InterruptedException e) {}
13        System.out.println("Agent " + agentName + " delivered Order #" + orderId);
14    }
15 }
16
17 public class FoodDeliverySystem {
18     public static void main(String[] args) {
19         ExecutorService deliveryAgents = Executors.newFixedThreadPool(3);
20         for (int i = 1; i <= 10; i++) {
21             deliveryAgents.execute(new DeliveryTask(i));
22         }
23         deliveryAgents.shutdown();
24     }
25 }
```

The console output shows the execution of the program:

```
<terminated> FoodDeliverySystem [Java Application] C:\Program Files\Java\jdk-26.0.1\bin\javaw.exe (2 May 2026, 1:31:12 pm - 1:31:20 pm elapsed 0:00:08) [pid: 11424]
Agent pool-1-thread-2 is picking up Order #2
Agent pool-1-thread-3 is picking up Order #3
Agent pool-1-thread-1 is picking up Order #1
Agent pool-1-thread-1 delivered Order #1
Agent pool-1-thread-1 is picking up Order #4
Agent pool-1-thread-2 delivered Order #2
Agent pool-1-thread-2 is picking up Order #5
Agent pool-1-thread-3 delivered Order #3
Agent pool-1-thread-3 is picking up Order #6
Agent pool-1-thread-3 delivered Order #6
Agent pool-1-thread-3 is picking up Order #7
Agent pool-1-thread-2 delivered Order #5
Agent pool-1-thread-2 is picking up Order #8
```